Parallel Scaling

Ana Costa Pereira, Joshua Davies

26/06/2026

1. SortBot Merging
2. SortBot Bottlenecks and changes
3. Changes in the SortBot Merge
4. SortBot Locking

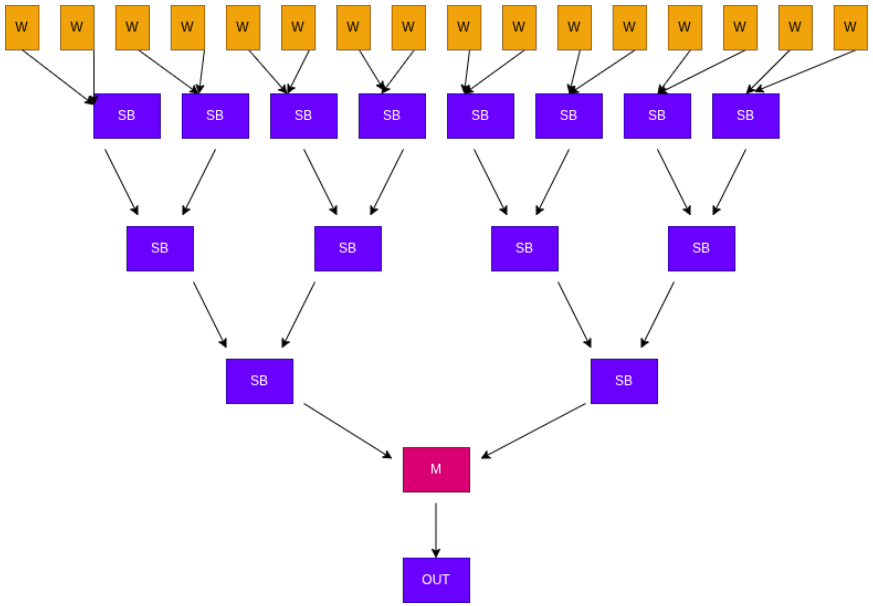
- Parallel scalings: study of scaling at higher thread number of tform;

Outline of talk

- Overview of sorting process in parallel at the SortBot level;
- Benchmarks performence at different thread count;
- Changes implemented at the level of SortBots and new benchmark results;

SortBot Merging

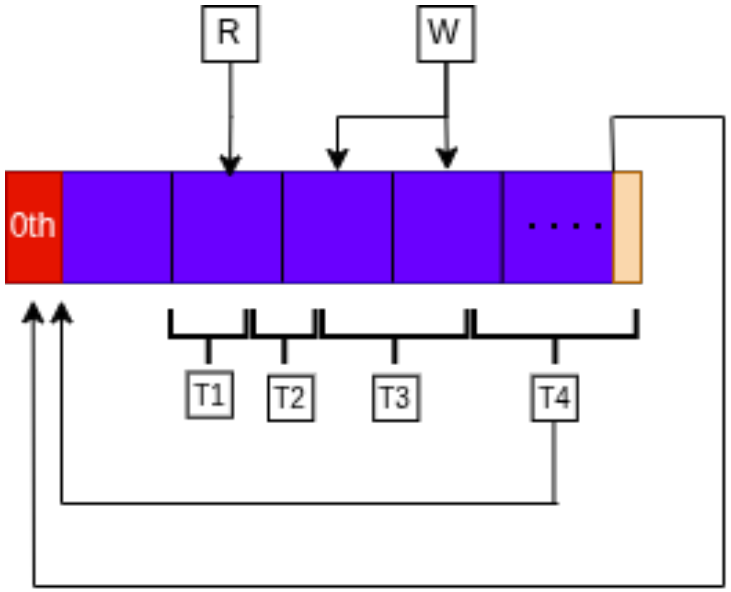
- Master $\longrightarrow$ allocations for the workers;
- Worker $\longrightarrow$ independent stream of sorted terms (identical terms live consecutively in memory);
- N workers $\longrightarrow$ $N-2$ SortBots are created;
- SortBots routine starts;



- Master performs the final merge and writes output.

SortBot: Overlaping of terms

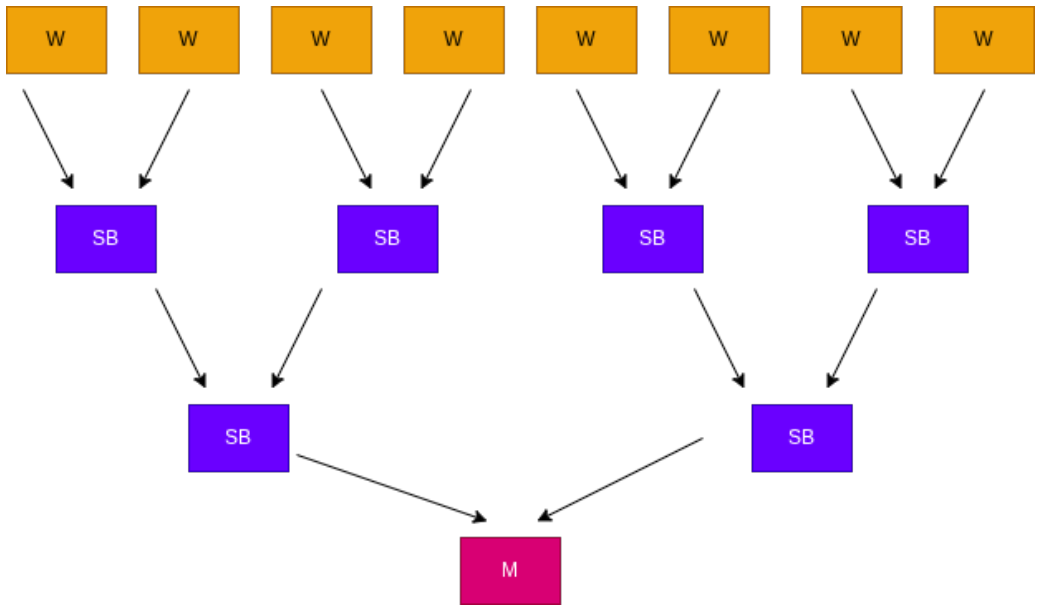
- SortBot $\longrightarrow$ blocks;



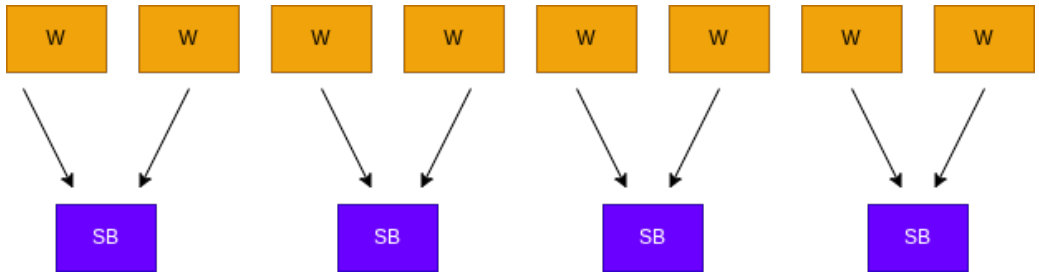
- Terms are allowed to overlap between blocks;
- 0th block - the overlap in the end of term is copied to 0th block so that the terms are contiguous.

SortBots Bottlenecks

- Bottlenecks: reading of the input with **division of terms** of workers to SortBots; inverse tree at the end with the **writing to the file by the Master**;

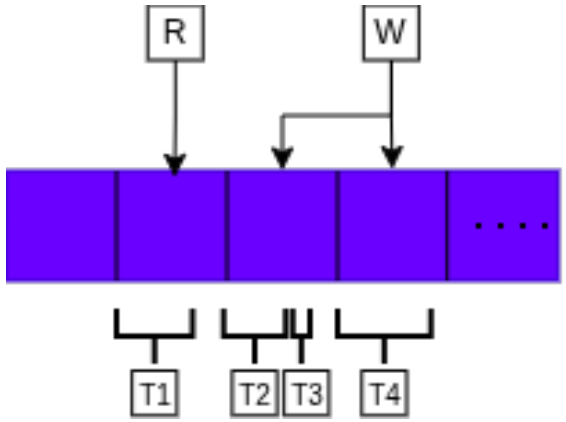


Bottlenecks



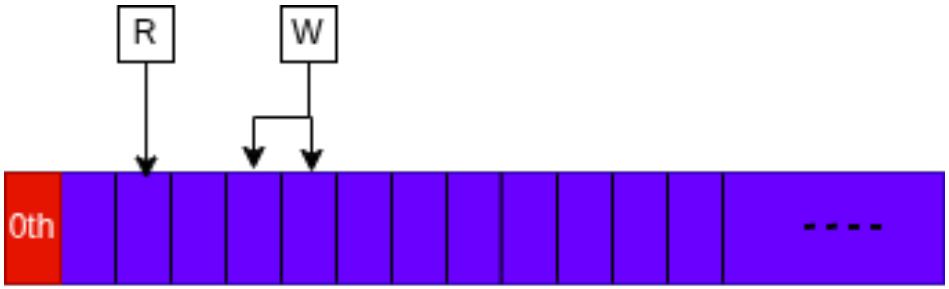
- Default buffer sized increased $\longrightarrow$ bigger SortBots;
- overlaping from worker fits into **single block** $\longrightarrow$ SortBotTree mechanism does not paralelize
- If a **term** is too big for one block: Worker fills the SortBots all the way up to the end of the block, **overlaping** to the next;
- Once a block is completely full, it reaches the top and moves on to the next block

SortBot Block size



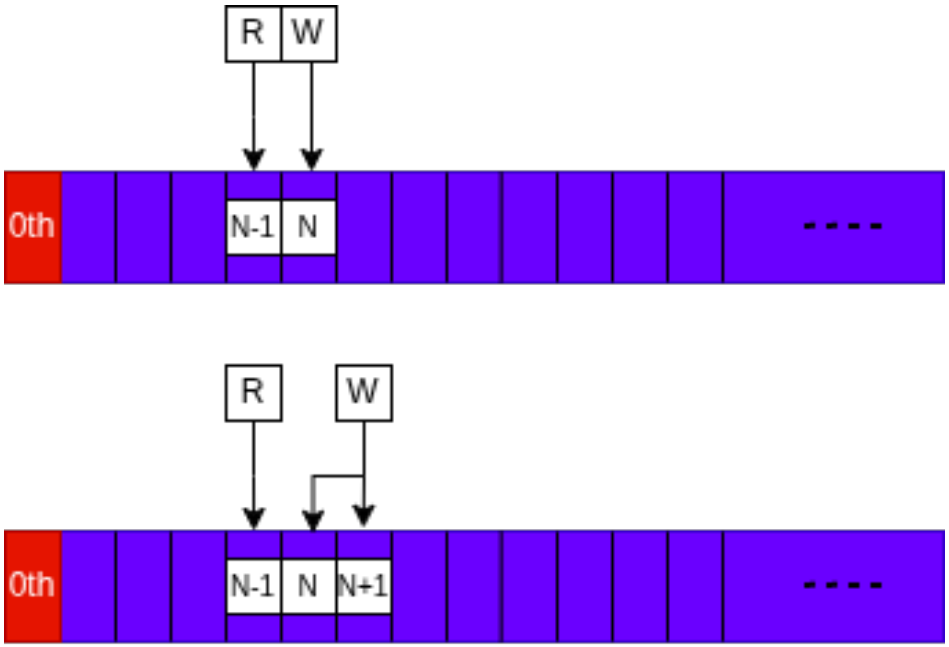
- Decrease the size of the allocations for the SortBots: blocks are made smaller;
- Whole terms must fit into the block: if it does not fit, it goes into the next block;
- 0th block is obsolete.

Locking



- Writting thread always locks pair of locks - the current thread and the previous thread;
- Reading threads locks the block where it is reading from;

SortBot: Locking

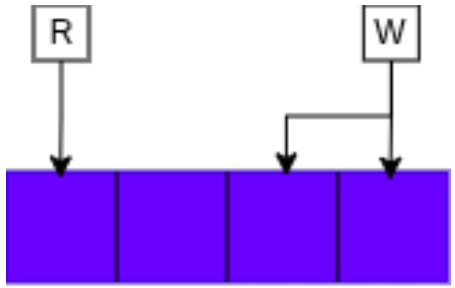


- Worker (W) detects if there is a Reader (R) waiting for them;
- W filling the blocks tries to obtain a lock of the block before the current fill block;
- If W does not obtain the lock, R thread already has the lock - very soon will be done and ready for more terms;
- The W moves to the next block;

SortBot variables

- The number of blocks in each SORTBOT has to be at least 4 (due to the locking mechanism) - which corresponds to a minumum of 8 for **NUMBEROFBLOCKSINSORT**;
- There ia a minimum number of terms that have to fit the block: **MINNUMBEROFTERMS**;
- Make sure a minimum number of terms is written: never make empty block - **MINWRITENUMBEROFTERMS**;

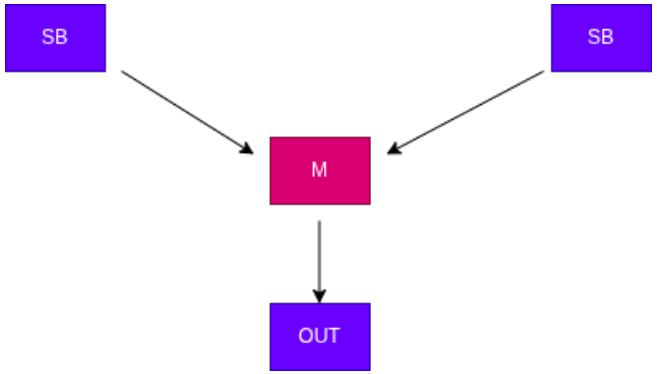
- NUMBEROFBLOCKSINSORT $= 8$
- MINNUMBEROFTERMS $= 1$
- MINWRITENUMBEROFTERMS $= 1$



Some results after the changes

- No decrease in performance for any tests;
- Performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l

Bottlenecks



- End of the tree: Master performs last sorting as well as writting of terms into the output;
- Performance could be increased by having one last stage of sorting of tha last 2 SortBots and Master only writting;

Adition of new SortBot

- Add a new SortBot at the end of the merging tree to do the sort of the streams that comes from last 2 SortBots;
- Master only does the writting.